Acta de Constitución del Proyecto

SISTEMA DE GESTIÓN DE PRODUCCIÓN Y VENTA DE POLLOS DE ENGORDE

Fecha: 22/10/2026

Gabriela Poveda, Dylan Mancipe y Juan Diego López
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte

TABLA DE CONTENIDO

Historial de Versiones 3
Información del Proyecto 3
Datos 3
Propósito y Justificación del Proyecto 3
Descripción del Proyecto y Entregables 4
Selección del Motor de Base de Datos Relacional 7
Método 7
Criterios y pesos 8
Matriz de decisión 8
Justificación por criterio 8
Análisis de sensibilidad 9
Requerimientos de alto nivel 9
Requerimientos del producto 9
Requerimientos del proyecto 10
Objetivos 11
Supuestos y Restricciones 12
Riesgos iniciales de alto nivel 13
Cronograma de hitos principales 14
Presupuesto estimado 14
Lista de Interesados (stakeholders) 15
Requisitos de aprobación del proyecto 15
Asignación del Gerente de Proyecto y nivel de autoridad 16
Gerente de Proyecto 16
Niveles de autoridad 16
Equipo del Proyecto 17
Referencias 17

Historial de Versiones

Fecha	Versión	Autor	Organización	Descripción
22/10/2026	1	Gabriela Poveda, Dylan Mancipe, Juan Diego López	LOS POLLOS HERMANOS	Primera versión del acta: propósito, justificación, descripción, entregables, selección del RDBMS, requerimientos, objetivos, supuestos, riesgos, cronograma, presupuesto, interesados, gerente, autoridad y equipo.
11/11/2026	2	Juan Diego López	LOS POLLOS HERMANOS	Ajuste del cronograma de hitos al calendario oficial (entrega del proyecto el 11/11/2026 y sustentación el 19/11/2026), presupuesto recalculado a 1 mes e infraestructura real: dos máquinas virtuales y el backend en el equipo anfitrión. Se retiran el diagrama de Gantt (está en el documento de cronograma) y la sección de aprobaciones.

Información del Proyecto

Datos

Empresa / Organización	Los Pollos Hermanos
Nombre del proyecto	Sistema de Gestión de Producción y Venta de Pollos de Engorde
Fecha de preparación	22/10/2026
Cliente	Ing., Esp., M.Ed. Christian Felipe Duarte
Patrocinador principal	Los Pollos Hermanos
Gerente de proyecto	Dylan Mancipe

Propósito y Justificación del Proyecto

Propósito:

El propósito del proyecto es diseñar, implementar y desplegar un sistema de información web que centralice la producción y la venta de una finca avícola de pollos de engorde, desde el ingreso de cada lote hasta el cobro de la venta, para conocer el costo real y la rentabilidad de cada lote.

El sistema contará con una base de datos relacional en PostgreSQL, que concentra la lógica de negocio mediante procedimientos almacenados, funciones y triggers, y con una base de datos documental en MongoDB para un único módulo: el seguimiento diario del crecimiento de cada lote. El backend se desarrollará en Java con Spring Boot, que expone una API REST, y la interfaz en Angular; las bases de datos se desplegarán en máquinas virtuales Linux y el backend en el equipo anfitrión.

Justificación:

En Colombia, muchas granjas avícolas pequeñas y medianas registran lotes, alimento, medicamentos, ventas y pagos en cuadernos u hojas de cálculo separadas. Cruzar esa información a mano produce demoras, errores y decisiones tardías sobre compra de alimento, fecha de venta y cobro de cartera.

Desde lo académico, el dominio exige un modelo de al menos 8 entidades con relaciones 1:1, 1:N y N:M, reglas de negocio que justifican procedimientos, funciones y triggers, y un seguimiento diario de estructura variable que justifica el uso de MongoDB, lo que permite aplicar de forma integrada lo visto en Bases de Datos I y II.

Descripción del Proyecto y Entregables

La empresa Los Pollos Hermanos desarrolla un sistema web para una finca avícola con una sola sede y varios galpones. El sistema cubre ocho módulos: galpones y lotes; alimentación por etapas (preiniciación días 1 a 7, iniciación 8 a 21, engorde 22 a 35 y finalización desde el día 36 hasta la venta); sanidad, incluidas las vitaminas de la primera semana; gastos, incluido el costo del beneficio (sacrificio y pelado de las aves); personal; comercial (clientes, pedidos por lote y ventas); cartera y entregas, con pagos parciales y pago en línea mediante el ambiente de pruebas de una pasarela de pagos; y seguimiento diario del crecimiento, que es el único módulo en MongoDB.

Cada lote tiene una mortalidad esperada de referencia del 5% de las aves iniciales. Aun así, el usuario debe registrar diariamente en PostgreSQL las aves que mueren: la cantidad disponible se calcula con la mortalidad real registrada y el sistema alerta cuando la mortalidad acumulada de un lote supera el 5%.

Costo del beneficio. Antes de la venta, las aves se preparan mediante el beneficio: sacrificio, escaldado, desplume y evisceración. Lo que el sistema controla es el costo de este proceso: por cada beneficio se registran el lote, la fecha, la cantidad de aves beneficiadas y el costo por ave, y el sistema calcula el costo total del beneficio. Ese costo se suma al costo del lote, entra en el costo por ave vendida y en la rentabilidad del lote, y permite comparar el precio de venta por kilo con lo que realmente costó producir y preparar cada pollo.

Costo del lote. El sistema calcula el costo real de cada lote como la suma de cinco componentes: los pollitos de un día; la alimentación, valorada como los bultos consumidos por

el costo promedio ponderado del bulto según las compras de cada tipo de alimento; las vitaminas y medicamentos, valorados como la cantidad aplicada por el costo unitario de compra del producto; el beneficio; y los demás gastos del lote (mano de obra, servicios públicos y transporte). Con ese costo se obtienen el costo por ave vendida y la rentabilidad del lote.

Uso de la base de datos no relacional (MongoDB). MongoDB se usa en un solo módulo: el seguimiento diario del crecimiento de cada lote. Cada documento corresponde a un lote en un día y guarda el peso promedio de las aves, la temperatura y la humedad del galpón y las observaciones del operario; algunos días incluye además un muestreo de peso (aves pesadas y uniformidad) o una novedad del galpón, como cama húmeda o falla en bebederos. Se elige una base documental porque la estructura de estos registros cambia de un día a otro, lo que en una tabla relacional obligaría a muchas columnas vacías o a tablas adicionales, y porque son registros que solo se agregan y se analizan en conjunto, lo que se aprovecha con agregaciones como la curva de crecimiento por semana, la ganancia diaria de peso y la comparación con la curva de referencia.

Concordancia entre las dos bases. Para que ambas bases concuerden, ningún dato se guarda en las dos. La mortalidad, el consumo de alimento, las aplicaciones sanitarias, los costos y todo lo transaccional se registran solo en PostgreSQL; MongoDB guarda solo el seguimiento del crecimiento. Cada documento se vincula al lote por su código, y el backend solo guarda un seguimiento si ese lote existe en PostgreSQL y está activo. La edad del lote se calcula con la fecha de ingreso registrada en PostgreSQL, de modo que ambas bases manejan el mismo lote, el mismo día y la misma edad.

Las reglas de negocio, como la capacidad del galpón, el único lote activo por galpón, el alimento según la edad del lote, los cargos autorizados por tipo de registro y el cálculo de saldo y cartera vencida, se implementan dentro de PostgreSQL con control transaccional y manejo de excepciones.

Fuera del alcance: facturación electrónica ante la DIAN e integración con sistemas contables; operación en varias sedes o monedas; liquidación de nómina; control de existencias de vitaminas y medicamentos (se registran sus compras y costos, pero no el inventario); captura automática desde básculas o sensores; aplicación móvil y despliegue en producción. La pasarela de pagos opera solo en su ambiente de pruebas, sin dinero real.

El proyecto usa las siguientes tecnologías:

Capa	Tecnología	Uso en el proyecto
Base de datos relacional	PostgreSQL (PL/pgSQL)	Datos transaccionales y lógica de negocio: procedimientos, funciones, triggers y transacciones
Base de datos NoSQL	MongoDB	Único módulo: seguimiento diario del crecimiento de cada lote, con CRUD, consultas complejas y agregaciones

Capa	Tecnología	Uso en el proyecto
Backend	Java con Spring Boot	API REST; conexión a PostgreSQL con Spring Data JPA y JdbcTemplate, y a MongoDB con Spring Data MongoDB; integración con la pasarela de pagos
Frontend	Angular (con Node.js y npm)	Interfaz web en español; Node.js se usa como entorno de desarrollo y compilación de Angular
Pagos en línea	Pasarela de pagos en ambiente de pruebas (sandbox)	Pagos simulados de las ventas, sin dinero real
Infraestructura	Dos máquinas virtuales con Ubuntu 26.04 (VirtualBox): Desktop mínima para PostgreSQL y Server para MongoDB	PostgreSQL y MongoDB en las máquinas virtuales; el backend y nginx corren en el equipo anfitrión
Control de versiones	Git y GitHub	Repositorio con ramas por funcionalidad y evidencia del trabajo de los tres integrantes
Modelado	Draw.io y Oracle SQL Developer Data Modeler	Modelos E-R en notación Chen y Crow's Foot, y modelo relacional

El proyecto produce los siguientes entregables:

Grupo	Entregable	Herramienta / formato
Gestión	Metodología de trabajo y su justificación	Word
Gestión	Requerimientos funcionales y no funcionales, con priorización y matriz de trazabilidad	Word / Excel
Gestión	Cronograma con alcance, hitos, estimación de tiempos y entregables parciales (diagrama de Gantt)	Excel / herramienta de Gantt
Gestión	Roles y matriz RACI	Excel
Gestión	Matriz de riesgos (probabilidad por impacto) con estrategias de mitigación	Excel
Gestión	Informe ejecutivo final (máximo 5 páginas sin anexos)	Word
Fase 1	Documento de propuesta (versión actualizada)	Word
Fase 1	Documento de supuestos (versión actualizada)	Word
Fase 1	Modelo E-R en notación Peter Chen	Draw.io
Fase 1	Modelo E-R extendido (si aplica)	Draw.io
Fase 1	Modelo E-R en notación Crow's Foot	Oracle SQL Developer Data Modeler

Grupo	Entregable	Herramienta / formato
Fase 1	Modelo relacional en notación Crow's Foot	Oracle SQL Developer Data Modeler
Fase 1	Diccionario de datos	Word / Excel
Fase 1	Documento SQL de reportes	Script .sql
Fase 2	Análisis de selección del sistema operativo	Word
Fase 2	Implementación de la máquina virtual	VM Ubuntu 26.04 (Desktop mínima y Server)
Fase 2	Análisis de selección del RDBMS	Word
Fase 2	Implementación del RDBMS	PostgreSQL
Fase 2	Scripts de creación, inserción y lógica (procedimientos, funciones, triggers, transacciones)	Scripts .sql
Fase 2	Modelo documental de MongoDB	Word + JSON de ejemplo
Fase 2	Implementación de MongoDB (CRUD, consultas complejas, agregaciones)	Scripts mongosh
Fase 2	Conexión de la aplicación con PostgreSQL	Spring Boot (Spring Data JPA) + Angular
Fase 2	Conexión de la aplicación con MongoDB	Spring Boot (Spring Data MongoDB) + Angular
Fase 2	Repositorio Git con ramas e historial colaborativo	GitHub
Fase 2	Video demostrativo (máximo 15 minutos, con los tres integrantes)	Video

Selección del Motor de Base de Datos Relacional

Decisión: el proyecto usará PostgreSQL como RDBMS. Obtuvo 4,85 de 5 en una matriz de decisión ponderada frente a las otras tres opciones que permite el enunciado (MySQL, MariaDB y SQL Server), y mantiene el primer lugar en los tres escenarios de sensibilidad evaluados.

Método

Se aplicó un análisis multicriterio por suma ponderada, con pesos asignados según la importancia de cada criterio para los objetivos del proyecto, en línea con los métodos de decisión multicriterio descritos por Saaty [1] y Triantaphyllou [2]. Cada alternativa se calificó de 1 (muy deficiente) a 5 (excelente) en cada criterio, y el puntaje final es la suma de cada calificación multiplicada por su peso.

Los pesos más altos se asignaron a la lógica procedural y al control transaccional, porque son el núcleo técnico del proyecto y equivalen al 20% de la rúbrica de evaluación (procedimientos, funciones, triggers, transacciones, excepciones, seguridad e integridad).

Criterios y pesos

Cód.	Criterio	Qué evalúa	Peso
C1	Lógica procedural	Procedimientos, funciones y tipos de triggers disponibles	25%
C2	Transacciones y excepciones	COMMIT/ROLLBACK en procedimientos, manejo de errores, DDL transaccional	20%
C3	Integridad declarativa	CHECK, dominios, tipos enumerados e índices parciales	15%
C4	Cercanía a PL/SQL	Transferencia de lo aprendido en clase con Oracle	10%
C5	Licencia y costo	Gratuidad y restricciones de uso	10%
C6	Despliegue en VM Linux	Soporte en Ubuntu Server y consumo de recursos	10%
C7	Integración con Java (Spring Boot)	Madurez del controlador JDBC y llamado a procedimientos desde Spring	5%
C8	Herramientas de modelado	Soporte en Data Modeler o Workbench, exigidos en la Fase 1	5%

Matriz de decisión

Criterio (peso)	PostgreSQL	MySQL	MariaDB	SQL Server
C1 Lógica procedural (25%)	5	3	3	4
C2 Transacciones y excepciones (20%)	5	3	3	4
C3 Integridad declarativa (15%)	5	3	3	4
C4 Cercanía a PL/SQL (10%)	5	3	4	2
C5 Licencia y costo (10%)	5	4	5	3
C6 Despliegue en VM Linux (10%)	5	5	5	3
C7 Integración con Java (5%)	5	5	5	5
C8 Herramientas de modelado (5%)	2	5	4	4
Puntaje ponderado	4,85	3,50	3,65	3,65

Justificación por criterio

- **C1. Lógica procedural.** PL/pgSQL ofrece triggers BEFORE, AFTER e INSTEAD OF, a nivel de fila y de sentencia [3]. En MySQL y MariaDB los triggers solo son por fila, no admiten COMMIT ni ROLLBACK y no pueden modificar la tabla que los disparó [4], lo que limita reglas como la actualización del estado del lote.
- **C2. Transacciones y excepciones.** Desde la versión 11, los PROCEDURE de PostgreSQL ejecutan COMMIT y ROLLBACK internamente; los bloques EXCEPTION actúan como

puntos de guardado implícitos y el DDL es transaccional [3]. En MySQL una sentencia DDL provoca un commit implícito.

- **C3. Integridad declarativa.** La regla de un solo lote activo por galpón se resuelve con un índice único parcial sobre los lotes activos, sin escribir código. MySQL no tiene índices parciales y obligaría a un trigger. Los DOMAIN y ENUM de PostgreSQL cubren los estados de lote, pedido y entrega.
- **C4. Cercanía a PL/SQL.** PL/pgSQL está inspirado en PL/SQL de Oracle, el lenguaje usado en el curso, por lo que el equipo reutiliza estructura de bloques, cursores y excepciones. T-SQL de SQL Server difiere bastante más.
- **C5. Licencia.** PostgreSQL se distribuye con la PostgreSQL License, libre y permisiva. SQL Server 2025 Express es gratuito pero limita cada base a 50 GB, el buffer pool a unos 1,4 GB y el cómputo a 4 núcleos; las ediciones Developer solo se licencian para desarrollo y pruebas [5].
- **C6 y C7. Despliegue e integración.** PostgreSQL está en los repositorios oficiales de Ubuntu Server y corre con pocos recursos en una VM. Su controlador JDBC, usado con Spring Data JPA y JdbcTemplate, permite invocar procedimientos con CALL y funciones con SELECT desde el backend en Spring Boot. En integración con Java los cuatro motores tienen controladores JDBC maduros, por lo que este criterio no los diferencia.
- **C8. Herramientas de modelado (desventaja).** MySQL Workbench solo trabaja con MySQL y Oracle SQL Developer Data Modeler no genera DDL nativo para PostgreSQL. Mitigación: los modelos Crow's Foot se elaboran en Data Modeler y el DDL de PostgreSQL se escribe y versiona manualmente en los scripts de creación de la Fase 2.

Análisis de sensibilidad

Para comprobar que la decisión no depende de los pesos elegidos, se recalculó la matriz en dos escenarios alternativos. PostgreSQL conserva el primer lugar en todos.

Escenario	PostgreSQL	MySQL	MariaDB	SQL Server
Base (pesos de la tabla de criterios)	4,85	3,50	3,65	3,65
Pesos iguales (12,5% cada criterio)	4,63	3,88	4,00	3,63
Modelado reforzado (C1 15%, C8 15%)	4,55	3,70	3,75	3,65

Incluso en el escenario que más favorece a las herramientas de modelado, la ventaja de PostgreSQL en lógica procedural, transacciones e integridad compensa su única desventaja.

Requerimientos de alto nivel

Requerimientos del producto

Requerimientos:

- **Galpones y lotes:** el sistema debe registrar galpones y lotes con el costo de los pollitos de un día, controlar la capacidad del galpón, permitir un solo lote activo por galpón y gestionar la transición de estados del lote.
- **Mortalidad y disponibilidad:** el usuario debe registrar la mortalidad diaria de cada lote; el sistema calcula la cantidad disponible con esa mortalidad real y alerta cuando la mortalidad acumulada supera el 5% esperado.
- **Alimentación por etapas:** el sistema debe registrar compras de alimento por bultos con su costo unitario y el consumo por lote, validando el inventario y la etapa según la edad del lote, y valorar ese consumo al costo promedio ponderado del bulto.
- **Sanidad:** el sistema debe registrar las compras de vitaminas y medicamentos con su costo unitario y las aplicaciones por lote, con un encargado de sanidad como responsable, y cargar al lote el costo de cada aplicación.
- **Gastos y costo por lote:** el sistema debe registrar gastos por lote o generales de la finca y calcular el costo total de cada lote.
- **Costo del beneficio:** el sistema debe registrar por lote el costo de preparar las aves para la venta (cantidad de aves beneficiadas por costo por ave), sumarlo al costo total del lote e incluirlo en el costo por ave vendida y en la rentabilidad.
- **Pedidos y ventas:** el sistema debe gestionar clientes, pedidos con aves de uno o varios lotes, y ventas generadas desde pedidos confirmados.
- **Pagos y cartera:** el sistema debe registrar pagos parciales, recibir pagos en línea mediante el ambiente de pruebas de una pasarela de pagos, calcular el saldo e identificar la cartera vencida.
- **Entregas:** el sistema debe registrar una entrega por venta, con un transportista como responsable.
- **Seguimiento diario:** el sistema debe registrar y consultar en MongoDB el peso promedio, las condiciones del galpón y las observaciones diarias de cada lote activo, con agregaciones de la curva de crecimiento.
- **Reportes:** el sistema debe generar reportes de costo y rentabilidad por lote, con el desglose de alimentación, sanidad, beneficio y demás gastos, además de ventas por cliente y estado de cartera.
- **Seguridad e integridad:** las reglas de negocio deben aplicarse en PostgreSQL con transacciones y excepciones, y la aplicación debe conectarse con un rol de privilegios mínimos.
- **Interfaz en Angular:** la interfaz debe desarrollarse en Angular, en español, y mostrar mensajes claros cuando la base de datos rechace una operación.

Requerimientos del proyecto

- Elaborar los documentos de gestión: metodología, requerimientos con priorización y trazabilidad, cronograma con hitos (diagrama de Gantt), roles con matriz RACI y matriz de riesgos.
- Entregar en la Fase 1 los modelos E-R en notación Chen y Crow's Foot, el modelo relacional normalizado hasta 3FN, el diccionario de datos y el documento SQL de reportes.
- Entregar en la Fase 2 los análisis de selección del sistema operativo y del RDBMS, los scripts de creación, inserción y lógica en PL/pgSQL, y la implementación de MongoDB con CRUD y agregaciones.
- Implementar la solución en una máquina virtual Linux, documentando su instalación y configuración.
- Mantener un repositorio en GitHub con ramas, commits descriptivos y participación equitativa de los tres integrantes.
- Elaborar un video demostrativo de máximo 15 minutos en el que participen los tres integrantes.
- Elaborar un informe ejecutivo final de máximo 5 páginas sin anexos, con referencias bibliográficas de revistas científicas y libros.

Objetivos

Objetivo general: desarrollar y desplegar en una máquina virtual Linux un sistema web que gestione lotes, alimentación, sanidad, clientes, pedidos, ventas, pagos y entregas de una finca avícola, con lógica de negocio en una base de datos PostgreSQL normalizada hasta 3FN e integración con MongoDB para el seguimiento diario del crecimiento de los lotes.

Objetivo	Indicador de éxito
Alcance	
Modelado: modelar el dominio con un mínimo de 8 entidades y relaciones 1:1, 1:N y N:M, normalizado hasta 3FN.	Modelos Chen, Crow's Foot y relacional aprobados por el docente, con 8 o más entidades y sin dependencias parciales ni transitivas.
Lógica en la base de datos: implementar en PL/pgSQL procedimientos, funciones y triggers que cubran las reglas de negocio de los supuestos.	Cada regla de negocio del documento de supuestos está implementada por al menos un objeto de base de datos y probada con un caso válido y uno inválido.
MongoDB: implementar el módulo de seguimiento diario del crecimiento con CRUD, consultas complejas y agregaciones.	Las cuatro operaciones CRUD y las agregaciones definidas funcionan desde la aplicación sobre datos de prueba.

Objetivo	Indicador de éxito
Aplicación: construir una aplicación web con backend en Java Spring Boot y frontend en Angular, conectada a PostgreSQL, MongoDB y la pasarela de pagos de prueba.	Demostración en la que cada módulo opera de punta a punta e invoca procedimientos almacenados.
Cronograma (Tiempo)	
Entregar la Fase 1, la Fase 2 y la sustentación en las fechas definidas por la asignatura.	100% de los entregables radicados en la fecha tope del cronograma de hitos.
Costo	
Ejecutar el proyecto dentro del presupuesto estimado.	Variación del costo real frente al estimado menor o igual al 10%.
Calidad	
Integridad: garantizar atomicidad y consistencia con transacciones y manejo de excepciones.	Ninguna operación fallida en las pruebas deja datos parciales; todos los errores se capturan con mensajes controlados.
Seguridad: proteger los datos con roles de base de datos y autenticación de usuarios.	La aplicación usa un rol sin permisos de DDL, las contraseñas se almacenan con hash y los datos de tarjeta nunca pasan por el sistema.
Rendimiento: responder las consultas y reportes habituales con rapidez.	Tiempo de respuesta menor a 2 segundos con el volumen de datos de prueba.
Otros	
Control de versiones: gestionar el código en GitHub con trabajo colaborativo.	Ramas por funcionalidad, commits descriptivos y aportes de los tres integrantes en el historial.
Video y sustentación: demostrar el funcionamiento y el dominio técnico del equipo.	Video de máximo 15 minutos en el que los tres integrantes explican y demuestran el sistema.

Supuestos y Restricciones

Restricciones:

Motor de base de datos: el RDBMS debe ser MySQL, PostgreSQL, MariaDB o SQL Server; Oracle no está permitido. La base de datos NoSQL debe ser MongoDB.

Sistema operativo: la solución debe implementarse en una máquina virtual con una distribución Linux o Unix.

Diseño de datos: el modelo relacional debe tener mínimo 8 entidades, relaciones 1:1, 1:N y N:M, y normalización hasta 3FN.

Equipo: el grupo está conformado por 3 integrantes, con participación equitativa demostrable en GitHub.

Tiempo: las entregas se rigen por el calendario académico de la asignatura.

Costo: el proyecto no cuenta con presupuesto monetario real; solo se usan recursos propios del equipo y herramientas gratuitas.

Entregables y sustentación: el video dura máximo 15 minutos con participación de todos, el informe ejecutivo tiene máximo 5 páginas sin anexos, no se aceptan proyectos incompletos y el plagio implica la anulación del proyecto.

Supuestos:

Se asume que los tres integrantes mantienen una disponibilidad semanal constante durante todo el semestre.

Se asume que al menos un equipo del grupo puede ejecutar la máquina virtual con PostgreSQL, MongoDB y el backend en Spring Boot al mismo tiempo.

Se asume que el sistema trabaja con datos simulados de una finca avícola, sin una finca real que lo valide.

Se asume una mortalidad de referencia del 5% por lote para planear compras y ventas; el valor real siempre proviene de los registros diarios del usuario.

Se asume que todas las herramientas usadas son gratuitas o tienen licencia académica, y que el ambiente de pruebas de la pasarela de pagos sigue disponible durante el proyecto.

Se asume que el docente revisa los entregables en las fechas definidas por la asignatura y comunica a tiempo los cambios. Las reglas del negocio avícola se detallan en el Documento de Supuestos.

Riesgos iniciales de alto nivel

- Baja disponibilidad o retiro de un integrante, lo que deja una carga de trabajo desigual y un historial de Git disparejo.

- Comprensión incorrecta de algún requerimiento del enunciado o del docente, que obligue a rehacer modelos o scripts.

- Lógica en PL/pgSQL más compleja de lo estimado (triggers, transacciones y excepciones), que retrase la Fase 2.

- Falla o pérdida de la máquina virtual o de los datos por no hacer respaldos ni subir los cambios al repositorio.

- Problemas de integración entre Angular, el backend en Spring Boot, PostgreSQL, MongoDB y la pasarela de pagos.

- Cambios o indisponibilidad del ambiente de pruebas de la pasarela de pagos.
- Conflictos de fusión o malas prácticas en Git que afecten la evidencia de trabajo colaborativo.
- Que algún integrante no tenga el dominio suficiente para defender el proyecto en la sustentación.

Cronograma de hitos principales

Las fechas son estimadas a partir del cierre del semestre y se ajustarán cuando el docente publique el calendario oficial de entregas. Se evitaron los festivos del 2 y el 16 de noviembre.

Hito	Fecha tope
Documentos de gestión (metodología, requerimientos, cronograma, RACI, riesgos)	29/10/2026
Entrega Fase 1: modelos, diccionario de datos y SQL de reportes	30/10/2026
Máquina virtual, PostgreSQL y MongoDB instalados	31/10/2026
Scripts de creación, inserción y lógica en PL/pgSQL	09/11/2026
Implementación de MongoDB	08/11/2026
Aplicación web integrada con ambas bases y la pasarela de pagos	09/11/2026
Pruebas integrales	10/11/2026
Video demostrativo	10/11/2026
Entrega Fase 2 e informe ejecutivo	11/11/2026
Sustentación	19/11/2026

Presupuesto estimado

El presupuesto es una valoración simulada del esfuerzo y los recursos del proyecto, con una duración de 1 mes (22 de octubre a 19 de noviembre de 2026). No implica desembolsos reales, conforme a la restricción de costo, y sirve como línea base para medir la variación definida en los objetivos.

Concepto	Detalle	Valor (COP)
Salarios	Gerente de proyecto (Dylan Mancipe): $3.500.000 mensuales por 1 mes	3.500.000
Salarios	Líder de base de datos e infraestructura (Juan Diego López): $3.200.000 mensuales por 1 mes	3.200.000
Salarios	Líder de frontend, calidad y documentación (Gabriela Poveda): $3.000.000 mensuales por 1 mes	3.000.000

Concepto	Detalle	Valor (COP)
Servicios	Internet: 3 hogares por $80.000 mensuales por 1 mes	240.000
Servicios	Energía eléctrica: 3 hogares por $50.000 mensuales por 1 mes	150.000
Servicios	Transporte para reuniones presenciales del equipo	120.000
Equipos	Depreciación de 3 computadores propios ($3.000.000 c/u, vida útil 36 meses) por 1 mes	250.000
Software	PostgreSQL, MongoDB Community, Java (OpenJDK), Spring Boot, Node.js, Angular, VirtualBox, Ubuntu Server, GitHub y sandbox de la pasarela	0
Subtotal		10.460.000
Fondo de imprevistos	10% del subtotal	1.046.000
Total		11.506.000

Lista de Interesados (stakeholders)

Nombre	Cargo	Departamento / División	Rama ejecutiva
Christian Felipe Duarte	Docente / Cliente	Facultad de Ingeniería / Ingeniería de Sistemas	Académica
Gabriela Poveda, Dylan Mancipe, Juan Diego López	Equipo del proyecto (Los Pollos Hermanos)	Tecnología de la Información	Desarrollo
Administrador de la finca	Usuario final	Administración	Operación
Operarios, encargado de sanidad y transportistas	Usuarios operativos	Producción y logística	Operación
Clientes (tiendas, restaurantes, personas)	Compradores	Ventas	Comercial
Proveedor de la pasarela de pagos	Proveedor tecnológico externo	Pagos en línea	Externa

Requisitos de aprobación del proyecto

- Cumplir todos los requisitos técnicos obligatorios del enunciado; no se aceptan proyectos incompletos.

- Entregar cada fase dentro del plazo definido por el docente a través de la plataforma de la asignatura.
- Presentar los modelos E-R, el modelo relacional en 3FN y el diccionario de datos coherentes entre sí.
- Demostrar procedimientos almacenados, funciones, triggers, transacciones y manejo de excepciones en PostgreSQL, invocados desde la aplicación web.
- Demostrar MongoDB con CRUD, consultas complejas y agregaciones, con la justificación de por qué esos datos se almacenan en NoSQL.
- Presentar un repositorio en GitHub con evidencia de participación equitativa.
- Entregar un video de máximo 15 minutos y sustentar con la participación de todos los integrantes.
- Entregar un trabajo original; el plagio implica la anulación del proyecto.

Asignación del Gerente de Proyecto y nivel de autoridad

Gerente de Proyecto

Nombre	Cargo	Departamento / División	Rama ejecutiva
Dylan Mancipe	Gerente del proyecto	Tecnología de la Información	Dirección general

Dylan Mancipe es designado gerente del proyecto por ser el integrante con mayor conocimiento del proceso productivo y comercial de los pollos de engorde. Este conocimiento del negocio le permite validar que los supuestos, las reglas implementadas en la base de datos y el funcionamiento del sistema correspondan a la operación real de una finca avícola.

Niveles de autoridad

Área de autoridad	Descripción del nivel de autoridad
Decisiones de personal (staffing)	Dylan Mancipe (gerente): asigna las tareas de cada integrante y redistribuye la carga cuando alguien se atrasa, buscando que los aportes en GitHub sean equitativos.
Gestión del presupuesto y sus variaciones	Gabriela Poveda: mantiene el presupuesto estimado y registra las variaciones. Si una variación supera el 10%, la reporta al gerente, que decide cómo ajustarla.
Decisiones técnicas	Juan Diego López decide sobre el modelo de datos, PL/pgSQL, la máquina virtual y el flujo de Git; Gabriela Poveda, sobre la interfaz en Angular y las pruebas; Dylan Mancipe, sobre las reglas del negocio avícola y MongoDB. Si dos áreas no se ponen de acuerdo, decide el gerente.

Área de autoridad	Descripción del nivel de autoridad
Resolución de conflictos	Dylan Mancipe (gerente): los conflictos se discuten en la reunión semanal del equipo y se buscan acuerdos por consenso; si no se logra, el gerente toma la decisión final y la deja registrada.
Ruta de escalamiento y limitaciones de autoridad	El escalamiento sigue este orden: integrante, líder del área, gerente y docente. Ningún integrante, incluido el gerente, puede modificar los requisitos obligatorios del enunciado ni el alcance aprobado en esta acta sin la aprobación del docente.

Equipo del Proyecto

Nombre	Rol dentro del proyecto	Departamento / División
Gabriela Poveda	Líder de frontend, calidad y documentación / Desarrolladora	Tecnología de la Información
Dylan Mancipe	Gerente de proyecto / Experto del negocio / Desarrollador	Tecnología de la Información
Juan Diego López	Líder de base de datos e infraestructura / Desarrollador	Tecnología de la Información

Referencias

[1] T. L. Saaty, “How to make a decision: The analytic hierarchy process,” *European Journal of Operational Research*, vol. 48, no. 1, pp. 9-26, 1990, doi: 10.1016/0377-2217(90)90057-I.

[2] E. Triantaphyllou, *Multi-Criteria Decision Making Methods: A Comparative Study*. Boston, MA, USA: Springer, 2000, doi: 10.1007/978-1-4757-3157-6.

[3] PostgreSQL Global Development Group, “PL/pgSQL: SQL procedural language,” en *PostgreSQL 18 Documentation*, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/18/plpgsql.html. [Consultado: 22-oct-2026].

[4] Oracle Corporation, “Stored objects,” en *MySQL 9.0 Reference Manual*, 2024. [En línea]. Disponible en: https://dev.mysql.com/doc/refman/9.0/en/stored-objects.html. [Consultado: 22-oct-2026].

[5] Microsoft, “Editions and components of SQL Server 2025,” *Microsoft Learn*, 2025. [En línea]. Disponible en: https://learn.microsoft.com/en-us/sql/sql-server/editions-and-components-of-sql-server-2025. [Consultado: 22-oct-2026].